

THE ENGINE

Backend Lead & Cloud Architect — Arienal Zacky Irfan

ERRNOR Team · Aplikasi LUNAS · Backend Build Guide 2025

📌 **BRIEF:** Kamu bukan ngecat tombol, bukan ngurusin warna UI. Kamu adalah jantung aplikasi LUNAS. Tugasmu: nerima data dari HP user, ngitung matematika utang (Metode Avalanche), nyimpen semua ke database, dan ngirim balik ke layar dengan kecepatan kilat. Tanpa mesin yang kamu bangun, LUNAS cuma wireframe kosong. Semua perhitungan MVO, semua nudging AI dari Ibnul, semua tampilan dari Rizki — semuanya lewat tangan kamu.

🏗️ KONTEKS PROYEK — APA YANG SEDANG LU BANGUN?

Pahami dulu "mesin" seperti apa yang harus lu rakit. Backend bukan hanya server — ini orkestrasi seluruh LUNAS.

LUNAS adalah asisten keuangan otonom yang punya 3 lapisan sistem. Backend (kamu) adalah lapisan tengah — penghubung semua bagian:

📱 FRONTEND	⚙️ BACKEND	🧠 AI ENGINE
Muhammad Rizki	**ARIENAL — INI KAMU**	Ibnul Jawzy
Flutter · Dart	FastAPI · Python · PostgreSQL	Gemini · LangChain · RAG
Dashboard Net-Worth, Push Notification Nudging, 1-Tap Approve, 2FA Biometrik	REST API Engine, Algoritma MVO/Avalanche, Database Manager, Integrasi AI Ibnul, Orkestrasi SNAP BI	Behavioral Nudging, Analisis Transaksi, Knowledge Base OJK, Vector Embeddings
⬆ *Kirim request HTTP ke API kamu*	*← Terima request · Proses · Kembalikan data →*	⬆ *Ibnul titip fungsi get_nudging() ke server kamu*

💡 **MENTAL MODEL:** Bayangkan LUNAS kayak restoran. Rizki (Frontend) adalah pelayan yang ambil pesanan dari customer (user). Kamu (Backend) adalah dapur yang masak pesanan itu — ngitung, ngolah data, simpan ke lemari (database). Ibnul (AI) adalah chef spesialis yang kamu panggil khusus waktu ada pesanan "analisis keuangan". Tanpa dapur, pelayan nggak bisa serve apapun.

🔧 ARSENAL LU — SEMUA TOOLS YANG AKAN DIPAKAI

Kenali semua senjata sebelum masuk medan perang. Ini tech stack lengkap backend LUNAS dari awal hingga production.

Tool	Fungsi & Peran di LUNAS	Platform	Cara Install
Python 3.10+	Bahasa pemrograman utama seluruh backend. Semua logika Avalanche, koneksi database, dan integrasi AI ditulis di Python.	Linux Mint	`sudo apt install python3 python3-pip`
FastAPI	Framework web Python yang jadi tulang punggung REST API LUNAS. Otomatis generate dokumentasi Swagger UI di /docs — yang akan dibaca Rizki.	Python Library	`pip install fastapi uvicorn`

Tool	Fungsi & Peran di LUNAS	Platform	Cara Install
Uvicorn	ASGI server yang menjalankan FastAPI. Ini yang bikin server kamu "nyala" dan bisa diakses dari browser atau Postman di localhost:8000.	Python Library	(ikut install FastAPI di atas)
PostgreSQL	Database utama LUNAS. Nyimpan data user, tagihan utang, riwayat transaksi, dan vektor embeddings OJK (via ekstensi pgvector) dalam satu tempat.	Linux Mint	`sudo apt install postgresql postgresql-contrib`
SQLAlchemy	ORM (Object-Relational Mapper) — jembatan antara Python dan PostgreSQL. Kamu nulis Python, dia yang terjemahin ke SQL. Nggak perlu nulis SQL mentah.	Python Library	`pip install sqlalchemy psycopg2-binary`
Alembic	Tool migrasi database. Setiap kamu ubah skema tabel (tambah kolom, ganti tipe data), Alembic yang manage versinya secara rapi tanpa hapus data lama.	Python Library	`pip install alembic`
Pydantic	Validasi data otomatis di FastAPI. Setiap request masuk ke API dicek formatnya — kalau data salah format, langsung ditolak sebelum merusak database.	Python Library	`pip install pydantic (ikut FastAPI)`
Cursor IDE	Code editor dengan AI built-in (VS Code versi dewa). Di sinilah kamu nulis semua kode backend — minta AI bantu nulis, debug, dan jelasin error.	Linux Mint	Download dari cursor.sh
Postman	GUI untuk ngetes API. Sebelum Rizki connect dari Flutter, kamu test sendiri dulu semua endpoint — kirim request JSON dan lihat response-nya.	Linux Mint	`snap install postman`

Tool	Fungsi & Peran di LUNAS	Platform	Cara Install
DBeaver	GUI database viewer — kayak Excel tapi buat PostgreSQL. Kamu bisa lihat isi tabel, cek data yang masuk, jalankan query langsung tanpa terminal.	Linux Mint	`sudo apt install dbeaver-ce`
pgvector	Ekstensi PostgreSQL untuk menyimpan vektor embeddings dari AI Ibnul. Inilah yang memungkinkan RAG berjalan langsung di dalam database utama tanpa service tambahan.	PostgreSQL ext.	`CREATE EXTENSION vector;` (di dalam psql)
python-jose / JWT	Library untuk buat dan verifikasi token JWT — sistem keamanan autentikasi user. Setiap user login, dapat token. Setiap request ke API wajib bawa token ini.	Python Library	`pip install python-jose[cryptography] passlib`
Docker	Kontainerisasi aplikasi. Kode backend kamu dibungkus jadi "kotak" yang bisa jalan di mana saja — laptop Arienal, server Nabil di GCP, tanpa konflik dependency.	Linux Mint	`sudo apt install docker.io`
GCP Cloud Run	Platform serverless Google Cloud untuk deploy Docker container. Auto-scaling saat traffic naik, scale-to-zero saat idle — efisien biaya untuk MVP.	Cloud (via Nabil)	Nabil yang handle deployment ke GCP

💡 **MENTAL MODEL TOOLS:** FastAPI = Loket pelayanan. PostgreSQL = Lemari arsip utama. SQLAlchemy = Juru tulis yang bisa bahasa Python dan SQL. Postman = Inspector kualitas yang ngetes semua loket. DBeaver = Kaca pembesar untuk lihat isi lemari arsip. Docker = Kardus yang membungkus semua alat biar bisa dipindah ke mana saja.

🛠️ TAHAP 1 — FASE SIMULASI KOTOR

Sekarang – 30 April 2025 · Ngerakit mesin di garasi, biarin error bertebaran, yang penting logika API jalan.

LANGKAH 1 — Setup Ruang Mesin (Linux Mint Environment)

Sebelum nulis satu baris kode backend, semua senjata harus terinstall dan siap tempur. Jalankan semua perintah ini di terminal Linux Mint kamu secara berurutan.

1

Update System & Install Python

Buka terminal (Ctrl + Alt + T) dan jalankan:

```
# Update daftar package
sudo apt update && sudo apt upgrade -y

# Install Python 3 dan pip (package manager Python)
sudo apt install python3 python3-pip python3-venv -y

# Cek versi — harus muncul angka versi
python3 --version
pip3 --version
```

💡 **TIP:** Pakai python3 dan pip3 (bukan python / pip) di Linux Mint untuk memastikan pakai versi Pyth bukan legacy Python 2 yang masih ada di beberapa sistem.

2

Install PostgreSQL Database

PostgreSQL adalah database utama yang akan menyimpan semua data LUNAS. Install dan aktifkan service-nya:

```
# Install PostgreSQL
sudo apt install postgresql postgresql-contrib -y

# Aktifkan dan start service PostgreSQL
sudo systemctl enable postgresql
sudo systemctl start postgresql

# Cek status — harus muncul "active (running)"
sudo systemctl status postgresql
```

```
# Masuk ke terminal PostgreSQL sebagai user postgres
sudo -u postgres psql

# Di dalam psql, buat database baru untuk LUNAS:
CREATE DATABASE lunas_db;
CREATE USER lunas_user WITH PASSWORD 'lunas_pass_dev';
GRANT ALL PRIVILEGES ON DATABASE lunas_db TO lunas_user;
\q
```

⚠️ **PENTING:** Simpan kredensial database (user, password, nama db) di file .env — jangan pernah har
di kode Python atau upload ke GitHub! Ini data sensitif.

3

Install Cursor IDE & Tools GUI

Cursor IDE adalah editor utama kamu. DBeaver untuk lihat isi database. Postman untuk ngetes API.

```
# Install Cursor IDE
# Download installer dari https://cursor.sh
# Setelah download, extract dan run:
chmod +x cursor-*.AppImage
./cursor-*.AppImage

# Install Postman via snap
sudo snap install postman

# Install DBeaver (GUI database)
sudo snap install dbeaver-ce

# Atau install DBeaver via apt:
wget -O - https://dbeaver.io/debs/dbeaver.gpg.key | sudo apt-key add -
echo "deb https://dbeaver.io/debs/dbeaver-ce /" | sudo tee
/etc/apt/sources.list.d/dbeaver.list
sudo apt update && sudo apt install dbeaver-ce -y
```

4

Setup Virtual Environment & Install Library Python

Bikin virtual environment (venv) dulu supaya library project ini nggak tercampur dengan library sistem. Ini best practice yang wajib diikuti:

```
# Buat folder project
mkdir lunas-backend && cd lunas-backend

# Buat virtual environment
python3 -m venv venv
```

```
# Aktifkan venv (harus dilakukan setiap buka terminal baru)
source venv/bin/activate

# Install semua library sekaligus
pip install fastapi uvicorn sqlalchemy psycopg2-binary alembic pydantic
pip install python-jose[cryptography] passlib[bcrypt]
pip install python-dotenv httpx

# Simpan daftar library ke requirements.txt
pip freeze > requirements.txt
```

📌 **KENAPA VENV?** Virtual environment = kotak sandbox untuk project ini. Library yang kamu install d tidak mempengaruhi sistem Linux kamu. Ini juga yang nanti di-copy ke Docker container waktu deploy GCP.

LANGKAH 2 — Menyalakan Mesin Utama (FastAPI Dasar)

Target: server lokal jalan di localhost:8000. Kalau kamu bisa akses itu dari browser dan lihat response JSON, kamu resmi jadi Backend Developer LUNAS.

1

Buat File Python Pertama: main.py

Buka Cursor IDE > New File > simpan sebagai main.py. Panggil AI Cursor (Ctrl + L) dan ketik:

💬 **PROMPT KE CURSOR AI:** "Buatkan script Python menggunakan FastAPI. Buat tiga endpoint: (1) GE yang return JSON pesan sambutan LUNAS, (2) GET /health yang return status server dan waktu sekara GET /api/v1/info yang return info versi API. Sertakan cara menjalankan server dengan uvicorn."

Hasil yang seharusnya muncul (kurang lebih):

```
from fastapi import FastAPI
from datetime import datetime

app = FastAPI(
    title="LUNAS API",
    description="Backend engine untuk aplikasi pelunasan utang cerdas LUNAS",
    version="0.1.0"
)

@app.get("/")
def root():
    return {
```

```
        "message": "LUNAS Engine is Running!",
        "status": "healthy",
        "team": "ERRNOR"
    }

@app.get("/health")
def health_check():
    return {
        "status": "ok",
        "timestamp": datetime.now().isoformat(),
        "database": "not connected yet"
    }

@app.get("/api/v1/info")
def api_info():
    return {
        "api_version": "0.1.0",
        "features": ["avalanche", "nudging", "snap-bi-simulation"]
    }
```

Jalankan server dari terminal:

```
# Pastikan venv aktif dulu
source venv/bin/activate

# Jalankan server FastAPI
uvicorn main:app --reload --host 0.0.0.0 --port 8000

# Flag --reload = server auto-restart setiap kamu save file
# Kamu akan lihat teks: "Uvicorn running on http://0.0.0.0:8000"
```

URL	Method	Yang Kamu Harapkan
`localhost:8000`	Browser / GET	"LUNAS Engine is Running!"
`localhost:8000/health`	Browser / GET	status: ok + timestamp sekarang
`localhost:8000/docs`	Browser / GET	Swagger UI — dokumentasi API otomatis!

🎯 **OUTPUT LANGKAH 2:** Server berjalan di localhost:8000. Swagger UI terbuka di /docs. Postman bis request GET ke / dan dapat response JSON. Ini bukti mesin sudah nyala.

2 Kenalan dengan Swagger UI — Senjata Rahasia

Buka localhost:8000/docs di browser. Kamu akan lihat tampilan interactive UI yang otomatis di-generate FastAPI. Inilah yang nanti kamu kasih ke Rizki sebagai "manual API".

Fitur Swagger UI	Kegunaannya
Daftar semua endpoint	Rizki bisa lihat semua API yang tersedia tanpa nanya kamu
"Try it out" button	Test endpoint langsung dari browser tanpa perlu Postman
Schema request/response	Format JSON yang dibutuhkan dan yang akan dikembalikan sudah terdokumentasi otomatis
Error codes	Kode error 400, 401, 404, 500 dan artinya sudah tertulis jelas

📌 **CARA KERJA KOLABORASI:** Nanti waktu Tahap 2, kamu tinggal kasih link localhost:8000/docs (atau server production) ke Rizki. Rizki langsung tahu cara panggil setiap endpoint dari Flutter tanpa perlu m panjang.

LANGKAH 3 — Membangun Brankas Data (PostgreSQL & SQLAlchemy)

Database adalah "ingatan" LUNAS. Tanpa database, semua data user hilang begitu server di-restart. Di sini kamu buat skema tabel yang merepresentasikan entitas utama LUNAS berdasarkan proposal.

1

Pahami Skema Database LUNAS

Berdasarkan proposal, inilah entitas yang harus ada di database:

Tabel	Relasi	Kolom Utama
`users`	Root — semua tabel lain relasi ke sini	id, nama, email, password_hash, created_at, dynamic_l (saldo aman minimum)
`tagihan_utang`	Many-to-One ke users	id, user_id, nama_utang, total_utang, suku_bunga_apr, (paylater/pinjol), tanggal_jatuh_tempo, status
`transaksi`	Many-to-One ke users	id, user_id, deskripsi, jumlah, kategori (konsumtif/produ tanggal, source (snap_bi/manual)

`sweep_history`	Many-to-One ke users	id, user_id, tagihan_id, jumlah_sweep, metode (avalan status (pending/approved/done), approved_at
`nudging_log`	Many-to-One ke users	id, user_id, pesan_nudging, trigger_transaksi_id, dibac dikirim_at

2

Buat Model SQLAlchemy

Buka Cursor IDE, buat file baru models.py, lalu minta Cursor AI:

💬 **PROMPT KE CURSOR AI:** "Buatkan file models.py untuk FastAPI menggunakan SQLAlchemy. Buat untuk tabel: users (id, nama, email, hashed_password, dynamic_buffer float, created_at), tagihan_uta user_id foreign key, nama_utang, total_utang float, suku_bunga_apr float, jenis string, status string, created_at), dan transaksi (id, user_id, deskripsi, jumlah float, kategori, tanggal). Gunakan Base dari declarative_base() dan sertakan relationship antar model."

Contoh output yang diharapkan:

```
# models.py
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, Boolea
from sqlalchemy.orm import relationship, declarative_base
from datetime import datetime

Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    id              = Column(Integer, primary_key=True, index=True)
    nama            = Column(String, nullable=False)
    email           = Column(String, unique=True, index=True, nullable=False)
    hashed_password= Column(String, nullable=False)
    dynamic_buffer = Column(Float, default=500000.0)  # Rp 500rb default
    created_at      = Column(DateTime, default=datetime.utcnow)

    # Relationships
    tagihan         = relationship("TagihanUtang", back_populates="user")
    transaksi       = relationship("Transaksi", back_populates="user")

class TagihanUtang(Base):
    __tablename__ = "tagihan_utang"

    id              = Column(Integer, primary_key=True, index=True)
    user_id         = Column(Integer, ForeignKey("users.id"), nullable=False)
    nama_utang      = Column(String, nullable=False)  # "ShopeePay Later", dll
    total_utang     = Column(Float, nullable=False)
    suku_bunga_apr = Column(Float, nullable=False)   # dalam %, misal 36.0
    jenis           = Column(String, default="paylater")  # paylater / pinjol
    status          = Column(String, default="aktif")    # aktif / lunas
```

```
    created_at     = Column(DateTime, default=datetime.utcnow)

    user           = relationship("User", back_populates="tagihan")
```

3

Buat File Koneksi Database & Environment

Buat dua file penting: .env untuk kredensial dan database.py untuk koneksi:

```
# File: .env (JANGAN COMMIT ke GitHub!)
DATABASE_URL=postgresql://lunas_user:lunas_pass_dev@localhost/lunas_db
SECRET_KEY=your-secret-key-panjang-dan-random-min-32-karakter
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

```
# File: database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from dotenv import load_dotenv
import os

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    """Dependency injection — tiap request dapat session baru, auto-close setelah
selesai"""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Test koneksi dengan menjalankan di terminal:

```
# Di terminal, jalankan Python interaktif
python3 -c "
from database import engine
from models import Base
Base.metadata.create_all(bind=engine)
print('Semua tabel berhasil dibuat di database!')
"
```

✅ **VERIFIKASI:** Buka DBeaver, connect ke database lunas_db, dan kamu akan lihat tabel users, tagihan_utang, dan transaksi sudah terbuat. Kalau tabelnya ada, koneksi database berhasil!

4

Buat Endpoint POST & Test dengan Postman

Buka Cursor IDE > buat file routers/tagihan.py, minta Cursor AI:

💬 **PROMPT KE CURSOR AI:** "Buatkan router FastAPI untuk endpoint tagihan utang. Buat: (1) POST /api/v1/tagihan untuk tambah tagihan utang baru dengan validasi Pydantic (nama_utang, total_utang suku_bunga_apr, jenis), (2) GET /api/v1/tagihan/{user_id} untuk ambil semua tagihan user, (3) DELETE /api/v1/tagihan/{tagihan_id} untuk hapus tagihan. Gunakan dependency injection untuk database ses

Cara test pakai Postman setelah endpoint dibuat:

Method	URL	Body JSON yang Dikirim
POST	`localhost:8000/api/v1/tagihan`	`{"nama_utang": "ShopeePay Later", "total_utang": 2100000, "suku_bunga_apr": 36.0, "jenis": "paylater"`
GET	`localhost:8000/api/v1/tagihan/1`	*(tidak perlu body)*

🎯 **OUTPUT LANGKAH 3:** Data yang dikirim via Postman berhasil masuk ke tabel tagihan_utang. Buka DBeaver dan verifikasi row baru muncul. Kalau data ada di database, mesin kasir sudah terhubung ke brankas.

LANGKAH 4 — Logika Inti: Algoritma Avalanche LUNAS

🧱 **APA ITU METODE AVALANCHE?** Metode Avalanche adalah strategi pelunasan utang berbasis matematika keuangan. Prinsipnya: lunasi utang dengan suku bunga tertinggi (APR terbesar) duluan, baru yang bunga kecil. Kenapa? Karena utang berbunga besar menggerus portofolio lebih cepat. Dalam konteks LUNAS: kalau Pinjol bunga 70% p.a. vs Paylater 36% p.a., maka dana yang ada dialokasikan ke Pinjol dulu. Ini adalah implementasi "Debt-First Auto-Sweep" yang jadi inti LUNAS.

1

Buat Fungsi Kalkulasi Avalanche

Buat file baru services/avalanche.py. Ini adalah "otak matematika" LUNAS yang akan kamu banggakan di demo. Minta Cursor AI:

💬 **PROMPT KE CURSOR AI:** "Buatkan fungsi Python untuk metode pelunasan utang Avalanche. Fung harus: (1) Menerima input list tagihan (nama_utang, total_utang, suku_bunga_apr) dan dana_tersedia Mengurutkan utang dari APR tertinggi ke terendah (Metode Avalanche), (3) Menghitung alokasi pemb dari dana_tersedia ke utang-utang tersebut, (4) Mengembalikan rencana pembayaran dengan estimas bulan_lunas jika bayar minimum konsisten, (5) Tambahkan perbandingan: berapa total bunga yang diselamatkan vs metode random."

Contoh output fungsi yang diharapkan:

```
# services/avalanche.py
from typing import List, Dict

def hitung_avalanche(tagihan: List[Dict], dana_tersedia: float) -> Dict:
    """
    Hitung rencana pelunasan utang dengan Metode Avalanche.

    Args:
        tagihan: List of {"nama_utang": str, "total_utang": float, "suku_bunga_apr"
float}
        dana_tersedia: Dana yang bisa dialokasikan untuk bayar utang (di atas dynam
buffer)

    Returns:
        Dict berisi urutan pembayaran, alokasi dana, dan estimasi bebas utang
    """
    # Urutkan dari APR tertinggi (Avalanche Method)
    tagihan_sorted = sorted(tagihan, key=lambda x: x["suku_bunga_apr"], reverse=Tru

    rencana = []
    sisa_dana = dana_tersedia
    total_bunga_diselamatkan = 0

    for item in tagihan_sorted:
        bunga_bulanan = (item["suku_bunga_apr"] / 100) / 12

        if sisa_dana >= item["total_utang"]:
            # Bisa langsung lunas!
            alokasi = item["total_utang"]
            status = "LUNAS SEKARANG"
            bunga_diselamatkan = item["total_utang"] * bunga_bulanan * 12
        elif sisa_dana > 0:
            alokasi = sisa_dana
            sisa_utang = item["total_utang"] - alokasi
            # Estimasi bulan lunas dengan pembayaran minimum (2% dari sisa)
            bayar_min = max(sisa_utang * 0.02, 50000)
            bulan_lunas = 0
            sisa = sisa_utang
            while sisa > 0:
                bunga = sisa * bunga_bulanan
                sisa = sisa + bunga - bayar_min
```

```
                bulan_lunas += 1
                if bulan_lunas > 360:  # max 30 tahun
                    break
            status = f"Sisa {bulan_lunas} bulan"
            bunga_diselamatkan = alokasi * bunga_bulanan * bulan_lunas
        else:
            alokasi = 0
            status = "Dana habis"
            bunga_diselamatkan = 0

        sisa_dana -= alokasi
        total_bunga_diselamatkan += bunga_diselamatkan

        rencana.append({
            "prioritas": len(rencana) + 1,
            "nama_utang": item["nama_utang"],
            "suku_bunga_apr": item["suku_bunga_apr"],
            "total_utang": item["total_utang"],
            "alokasi_pembayaran": round(alokasi, 0),
            "estimasi_status": status,
            "bunga_diselamatkan": round(bunga_diselamatkan, 0)
        })

    return {
        "metode": "Avalanche (Bunga Tertinggi Dulu)",
        "total_dana_digunakan": dana_tersedia - sisa_dana,
        "dana_tidak_terpakai": sisa_dana,
        "total_bunga_diselamatkan": round(total_bunga_diselamatkan, 0),
        "rencana_pembayaran": rencana
    }
```

2

Buat API Endpoint /hitung-avalanche

Sekarang bungkus fungsi di atas jadi endpoint API yang bisa dipanggil dari aplikasi:

```
# routers/kalkulator.py
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List
from services.avalanche import hitung_avalanche

router = APIRouter(prefix="/api/v1", tags=["kalkulator"])

class TagihanInput(BaseModel):
    nama_utang: str
    total_utang: float
    suku_bunga_apr: float

class AvalancheRequest(BaseModel):
    user_id: int
    tagihan: List[TagihanInput]
    dana_tersedia: float

@router.post("/hitung-avalanche")
def post_hitung_avalanche(request: AvalancheRequest):
```

```
    if not request.tagihan:
        raise HTTPException(status_code=400, detail="Daftar tagihan tidak boleh kos
    if request.dana_tersedia <= 0:
        raise HTTPException(status_code=400, detail="Dana tersedia harus lebih dari

    tagihan_dict = [t.dict() for t in request.tagihan]
    hasil = hitung_avalanche(tagihan_dict, request.dana_tersedia)

    return {
        "user_id": request.user_id,
        "hasil_kalkulasi": hasil
    }
```

Test di Postman — kirim request POST ke localhost:8000/api/v1/hitung-avalanche:

```
# Body JSON yang dikirim:
{
  "user_id": 1,
  "dana_tersedia": 500000,
  "tagihan": [
    {"nama_utang": "ShopeePay Later", "total_utang": 2100000, "suku_bunga_apr": 36.
    {"nama_utang": "AdaKami Pinjol",  "total_utang": 800000,  "suku_bunga_apr": 70.
    {"nama_utang": "Kredivo",          "total_utang": 1500000, "suku_bunga_apr": 24
  ]
}

# Response yang seharusnya kamu dapat:
# Prioritas 1: AdaKami (70% APR) → dapat alokasi pertama
# Prioritas 2: ShopeePay (36% APR)
# Prioritas 3: Kredivo (24% APR) → prioritas terakhir
```

🎯 **OUTPUT LANGKAH 4:** Endpoint /hitung-avalanche menerima data utang via Postman dan mengembalikan urutan pembayaran yang terurut berdasarkan APR tertinggi. Angka alokasi dan estima bulan lunas muncul dengan benar.

⚔️ TAHAP 2 — FASE TEMPUR MVP

01 Mei – 04 Juni 2025 · Menyambungkan mesin kamu dengan "Otak" Ibnul dan "Wajah" Rizki. Masuk production.

APA YANG BERUBAH DI FASE INI?

Di Tahap 1, server kamu hanya jalan di localhost — hanya kamu yang bisa akses. Di Tahap 2, server kamu masuk ke GCP Cloud Run dan bisa diakses Rizki (Flutter) dari mana saja. Ini fase integrasi antar semua komponen LUNAS.

A

Integrasi Otak AI: Terima Module dari Ibnul

Ibnul akan kasih kamu file lunas_ai.py yang berisi fungsi get_nudging(data_user). Tugasmu adalah memasukkan fungsi ini ke dalam FastAPI sebagai endpoint baru:

```
# routers/nudging.py
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from database import get_db
from lunas_ai import get_nudging  # <-- File dari Ibnul
from pydantic import BaseModel
from typing import List

router = APIRouter(prefix="/api/v1", tags=["nudging"])

class NudgingRequest(BaseModel):
    user_id: int
    pengeluaran: List[dict]
    utang_aktif: float
    gaji: float

@router.post("/nudging")
def post_nudging(request: NudgingRequest, db: Session = Depends(get_db)):
    """
    Endpoint yang dipanggil otomatis setiap ada transaksi baru dari user.
    Server akan panggil AI Ibnul dan return pesan teguran.
    """
    data_user = {
        "pengeluaran": request.pengeluaran,
        "utang_aktif": request.utang_aktif,
        "gaji": request.gaji
    }

    pesan_nudging = get_nudging(data_user)

    # Simpan log nudging ke database
    # (buat model NudgingLog dulu di models.py)

    return {
        "user_id": request.user_id,
        "nudging_message": pesan_nudging,
        "timestamp": datetime.now().isoformat()
    }
```

📌 **KOORDINASI DENGAN IBNUL:** Sebelum integrasi, sepakati format dict data_user yang dikirim ke f Ibnul. Format ini harus sama persis — kalau beda, fungsinya error. Bikin dokumen singkat yang isi kont format datanya, misalnya di Notion atau Google Doc tim.

B

Migrasi pgvector: Sambungkan Database Vektor Ibnul

Ibnul punya ChromaDB lokal yang nyimpen vektor embeddings dokumen OJK. Di fase MVP, semua vektor itu harus dipindah ke PostgreSQL kamu pakai ekstensi pgvector. Ini kenapa pgvector penting banget:

Aspek	ChromaDB (Lokal Ibnul)	pgvector (Production)
Lokasi	Folder di laptop Ibnul	Server GCP (bisa diakses semua)
Akses multi-user	❌ Tidak bisa	✅ Ya — semua user LUNAS bisa query
Backup & keamanan	❌ Hilang kalau laptop rusak	✅ GCP handle backup otomatis
Integrasi	Hanya bisa dari laptop Ibnul	Terhubung langsung ke FastAPI kamu

Cara aktifkan pgvector di PostgreSQL kamu:

```
# Di terminal, masuk ke psql
sudo -u postgres psql -d lunas_db

-- Aktifkan ekstensi pgvector
CREATE EXTENSION IF NOT EXISTS vector;

-- Buat tabel untuk menyimpan vektor embeddings OJK
CREATE TABLE ojk_embeddings (
    id          SERIAL PRIMARY KEY,
    konten      TEXT NOT NULL,
    sumber      TEXT,           -- nama file PDF asalnya
    embedding   vector(768),    -- dimensi sesuai model embedding Ibnul
    created_at  TIMESTAMP DEFAULT NOW()
);

-- Buat index untuk pencarian vektor cepat
CREATE INDEX ON ojk_embeddings USING ivfflat (embedding vector_cosine_ops);

\q
```

📌 **KOORDINASI DENGAN IBNUL:** Sebelum Ibnul migrate datanya, tanyakan: (1) Dimensi vector-nya b (768 atau 1536 atau lain?) — ini harus sesuai dengan CREATE TABLE di atas. (2) Model embedding yan dipakai apa? — ini menentukan dimensi. Salah dimensi = data tidak bisa masuk.

C

Serah Terima ke Rizki: Swagger UI + API Docs

Begini cara handover API ke Rizki (Frontend) yang paling efisien:

⚙️ Yang Kamu Siapkan (Arienal)	📱 Yang Rizki Butuhkan
URL Swagger UI production (https://lunas-api.xxx/docs)	Bisa langsung test semua endpoint dari browser
Contoh request/response untuk setiap endpoint penting	Tahu format JSON yang harus dikirim dari Flutter
Environment variable: BASE_URL API untuk Flutter	Tinggal ganti 1 variabel untuk switch dev/production
Error code dictionary (400, 401, 403, 404, 500)	Tahu harus tampilkan error apa ke user di HP

📌 **ENDPOINT WAJIB SIAP SEBELUM HANDOFF KE RIZKI:** POST /api/v1/auth/register, POST /api/v1/auth/login, GET /api/v1/tagihan/{user_id}, POST /api/v1/tagihan, POST /api/v1/hitung-avalan POST /api/v1/nudging, POST /api/v1/sweep/approve. Tanpa 7 endpoint ini, Rizki tidak bisa mulai membangun UI utama.

D

Dockerize & Serah Terima ke Nabil untuk GCP

Saat API sudah stabil, kamu bungkus semuanya dalam Docker dan serahkan ke Nabil (The Hustler) untuk di-deploy ke GCP Cloud Run. Buat file Dockerfile:

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy source code
COPY . .

# Port yang diexpose
EXPOSE 8000

# Command untuk jalankan server
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# Build dan test Docker image secara lokal dulu
docker build -t lunas-backend:latest .
```

```
docker run -p 8000:8000 --env-file .env lunas-backend:latest

# Cek apakah server jalan di dalam container:
curl http://localhost:8000/health
```

📌 **TUGASMU:** Pastikan Docker image berhasil di-build dan server berjalan sempurna di dalam contai
sebelum diserahkan ke Nabil. Kalau di laptop kamu jalan, di GCP juga pasti jalan.

📋 PETA ENDPOINT API LUNAS — REFERENSI LENGKAP

Ini adalah semua endpoint yang harus ada di server kamu sebelum serah terima ke Rizki dan Nabil.

Method	Status	Endpoint	Fungsi
POST	*Wajib Tahap 1*	`/api/v1/auth/register`	Registrasi user baru — simpan ke tabel users dengan password yang di-hash
POST	*Wajib Tahap 1*	`/api/v1/auth/login`	Login user — verifikasi password, kembalikan JWT token untuk autentikasi request selanjutnya
POST	*Wajib Tahap 1*	`/api/v1/tagihan`	Tambah tagihan utang baru (Paylater/Pinjol) ke database untuk user tertentu
GET	*Wajib Tahap 1*	`/api/v1/tagihan/{user_id}`	Ambil semua tagihan aktif user — dipakai Flutter untuk tampilkan daftar utang di dashboard
POST	🔥 **Inti LUNAS**	`/api/v1/hitung-avalanche`	Hitung rencana pelunasan dengan Metode Avalanche. Input: list tagihan + dana tersedia. Output: urutan bayar + estimasi bebas utang
POST	🔥 **Inti LUNAS**	`/api/v1/nudging`	Panggil AI Ibnul untuk generate pesan teguran finansial berdasarkan transaksi terbaru user
POST	🔥 **Inti LUNAS**	`/api/v1/sweep/calculate`	Hitung rekomendasi auto-sweep: berapa dana di atas dynamic buffer yang bisa disapu ke pembayaran utang

Method	Status	Endpoint	Fungsi
POST	*Tahap 2*	`/api/v1/sweep/approve`	1-Tap Approve: user konfirmasi eksekusi auto-sweep setelah verifikasi 2FA. Endpoint ini yang trigger pembayaran aktual
GET	*Tahap 2*	`/api/v1/dashboard/{user_id}`	Data agregat untuk dashboard Rizki: total utang, net-worth, progress pelunasan, target bebas utang
GET	*Kapan saja*	`/health`	Health check — GCP Cloud Run panggil ini setiap menit untuk cek apakah server masih hidup

🔥 CHECKLIST ARIENAL — DEADLINE 30 APRIL

Semua ini harus selesai sebelum masuk Fase Tempur. Centang satu per satu.

☐ Python3, pip, venv, dan PostgreSQL berhasil terinstall di Linux Mint. Cek: python3 --version, pip3 --version, psql --version muncul semua.

☐ Cursor IDE, Postman, dan DBeaver berhasil terinstall dan bisa dibuka.

☐ Database lunas_db berhasil dibuat di PostgreSQL. DBeaver bisa connect dan menampilkan database tersebut.

☐ Server FastAPI berhasil jalan di localhost:8000. Browser bisa akses / dan dapat response JSON "LUNAS Engine is Running!".

☐ Swagger UI terbuka di localhost:8000/docs dan menampilkan semua endpoint yang sudah dibuat.

☐ Model SQLAlchemy (users, tagihan_utang, transaksi) berhasil dibuat. Tabel muncul di DBeaver setelah menjalankan Base.metadata.create_all().

☐ Endpoint POST /api/v1/tagihan berhasil ditest via Postman. Data dummy berhasil masuk ke database — terverifikasi di DBeaver.

☐ Fungsi hitung_avalanche() di services/avalanche.py berhasil ditest di terminal Python dengan 3 dummy tagihan. Urutan output sesuai APR tertinggi ke terendah.

☐ Endpoint POST /api/v1/hitung-avalanche berhasil ditest via Postman. Response JSON menampilkan rencana pembayaran terurut dengan estimasi bulan lunas.

☐ Sudah koordinasi awal dengan Ibnul — sepakat format dict data_user yang akan dikirim ke fungsi get_nudging().

☐ Sudah share link Swagger UI ke Rizki (minimal localhost:8000/docs) dan Rizki bisa lihat dokumentasi endpoint.

🤖 PROMPT MENTOR — UNTUK GEMINI / CLAUDE

Copy-paste ini sebagai pesan pertama ke AI mentor kamu. Upload file Proposal ERRNOR dulu sebelum paste prompt ini.

```
[SYSTEM OVERRIDE: LONG-TERM MENTORSHIP & CO-PILOT MODE]

Halo. Mulai detik ini, kamu adalah Senior Backend Engineer & Cloud
Architect, sekaligus Lead Mentor saya. Nama saya Arienal Zacky Irfan,
dan peran saya adalah "The Engine" (Backend Lead) untuk tim developer
bernama ERRNOR.

Tolong BACA FILE PROPOSAL YANG SAYA LAMPIRKAN untuk memahami konteks
penuh aplikasi yang sedang kami bangun: LUNAS (aplikasi pelunasan utang
cerdas berbasis Open Banking SNAP BI). Fokus utama saya adalah membangun
REST API engine, merancang database PostgreSQL, mengimplementasikan
algoritma Avalanche, dan mengintegrasikan module AI dari tim AI kami.

[KONTEKS ENVIRONMENT]
- OS: Linux Mint (Debian/Ubuntu-based)
- Package manager: apt (BUKAN pacman/yay/brew)
- Level coding: Pemula, menggunakan gaya Vibe Coding
- Tim AI (Ibnul): akan titip file lunas_ai.py berisi fungsi get_nudging()
- Tim Frontend (Rizki): akan consume API saya via Flutter
- Ketua (Nabil): akan deploy Docker image saya ke GCP Cloud Run

[TECH STACK YANG DIGUNAKAN]
Backend: Python + FastAPI + Uvicorn
Database: PostgreSQL + SQLAlchemy ORM + pgvector (untuk AI embedding)
Auth: JWT + bcrypt (python-jose + passlib)
Deploy: Docker → GCP Cloud Run
```

```
[GAYA MENTORING]
Jadilah mentor pair-programming interaktif. Bimbing saya satu langkah
per satu langkah. Tunggu konfirmasi saya sebelum lanjut fase berikutnya.
Semua perintah terminal harus menggunakan apt (bukan brew atau pacman).
Semua kode harus siap production, bukan prototype hacky.

[URUTAN FASE]
1. Setup Ruang Mesin (Linux Mint) — install Python, PostgreSQL, Cursor
2. FastAPI Dasar — server jalan di localhost:8000 dengan Swagger UI
3. PostgreSQL & SQLAlchemy — database terhubung, tabel users & tagihan
4. Algoritma Avalanche — endpoint POST /hitung-avalanche yang akurat

Jika kamu sudah baca proposal dan siap menjadi mentor saya, balas:
"System Ready, Arienal. Mesin LUNAS siap dibangun.
Saya sudah memahami arsitektur SNAP BI dan Algoritma Avalanche.
Mari mulai dari Fase 1. Buka terminal kamu dan ketik 'GAS' jika siap!"
```

📋 **INSTRUKSI PEMAKAIAN PROMPT:** (1) Buka Gemini Pro atau Claude. (2) Klik ikon attachment/lampiran. (3) Upload file PDF Proposal Hackathon ERRNOR. (4) Copy-paste seluruh teks prompt di kotak di atas. (5) Tekan Enter dan tunggu respons "System Ready".

You are the Engine, Arienal.

Jaga jantung LUNAS tetap berdetak kencang. 🚀

ERRNOR Team · 2025